

# 芯鉴知微·硬件开发指导

Oct 10, 2026 · @gusuisui

## 目标与范围

硬件侧的下一个目标是：用一块真实的 ESP32 + DHT11，跑通“板上采集 → 上报平台 → 平台诊断 → 故障与恢复”的完整证据链，并留下可追溯的实测记录。

| 项目    | 当前状态                                                                  |
|-------|-----------------------------------------------------------------------|
| 选型    | ESP32-DevKitC V4 (ESP32-WROOM-32E) + Aosong 四针裸 DHT11 (2026-09-28 确定) |
| 固件    | 0.2.5，设备协议 V1；已编译，主机模拟测试通过                                            |
| 实验资料包 | DHT11 2.0.11 (测试草稿)                                                   |
| 实物验收  | 未执行：接线、烧录、真实读数、平台接入、故障与掉电均待实测                                         |

本指导面向第一次动手的成员，给出顺序和要点；逐条命令与记录表格见仓库里的 docs/experiments/dht11-internal-lab.md (内部实验指南) 和 docs/experiments/dht11-arrival-checklist.md (到货执行清单)。

## 硬件清单与接线

接线只有四根线加一个上拉电阻，必须在**断电状态**下接好并拍照留证。

| DHT11 引脚   | 接到 ESP32 | 说明                        |
|------------|----------|---------------------------|
| 脚 1 (VDD)  | 3V3      | 不要接 5V                    |
| 脚 2 (DATA) | GPIO4    | DATA 与 3V3 之间加 4.7kΩ 上拉电阻 |
| 脚 3 (NC)   | 不接       | 四针裸件的空脚                   |
| 脚 4 (GND)  | GND      | 与开发板共地                    |

器材：ESP32-DevKitC V4 开发板、四针裸 DHT11、4.7kΩ 电阻、面包板与杜邦线、能传数据的 USB 线。

- 传感器线尽量短，首个实验建议不超过 20cm。
- DHT11 是单总线器件，不是 I2C；两次读取间隔必须大于 2 秒，固件固定为 3 秒。
- 如果买到的是带底板的三针模块或其他版本，先对照手册复核引脚和数据格式，再上电。

## 开发环境与烧录

固件用 PlatformIO 构建，依赖版本全部锁定：Espressif32 7.0.1、Arduino core 3.20017.x、ArduinoJson 7.4.2。不要随意升级依赖，否则无法复现已测试过的构建。

1. 安装 PlatformIO Core（命令行 `pio`）或 VS Code 的 PlatformIO 插件。
2. 在仓库根目录编译：`pio run -d firmware/esp32_dht11`。没有配置文件也能编译，此时固件只在串口输出读数，不联网。
3. 插拔 USB 对照确认实际串口，再烧录：`pio run -d firmware/esp32_dht11 -t upload`。
4. 打开串口观察（115200 波特率）：`pio device monitor -d firmware/esp32_dht11`。
5. 仅对全新设备初始化一次 LittleFS 文件系统：先 `-t buildfs` 生成镜像，再按指南核对端口后 `-t uploadfs`。这个操作会清空设备上的缓存批次，不能用来“修复”上传问题。

每次构建都保存日志和固件的 SHA-256，写进到货清单。无硬件时可以先跑 `python scripts/test_firmware_host.py`，它用真实固件源码模拟文件、网络、重启和回执故障。

## 固件结构与关键配置

固件只有三个核心模块，联网所需的一切都写在一个不提交的本地文件里。

| 文件                                 | 作用                                                                   |
|------------------------------------|----------------------------------------------------------------------|
| <code>src/main.cpp</code>          | 主循环：定时触发采样、组装请求、联网上报与重试                                              |
| <code>src/dht11_reader.cpp</code>  | 按 Aosong V1.3 时序读 40 位帧，校验和与量程检查，失败只报 <code>DHT11_READ_FAILED</code> |
| <code>src/pending_store.cpp</code> | 把待发送批次持久化到 LittleFS，确认后才清理                                           |
| <code>include/secrets.h</code>     | 本地配置（从 <code>secrets.example.h</code> 复制，已被 Git 忽略）                  |

`secrets.h` 里要填的项：

- `XJ_WIFI_SSID` / `XJ_WIFI_PASSWORD`：实验用 Wi-Fi。
- `XJ_API_BASE_URL`：后端上报地址，形如 `https://<服务器>/api/v1/device/ingest`。
- `XJ_DEVICE_ID` / `XJ_DEVICE_TOKEN`：平台为这块设备创建的凭据。
- `XJ_EXPERIMENT_SESSION_ID`：学生在平台上开始实验后得到的会话 ID。
- `XJ_CA_CERT`：用 HTTPS 时填服务器证书；缺证书时固件拒绝发送。
- `XJ_ALLOW_INSECURE_HTTP`：只在隔离的本地测试服务器上设为 1，正式环境保持 0。

几个会影响判断的固件行为：

- **首帧不发布**：DHT11 返回的是上一次转换结果，上电后第一个有效帧只用于“预热”，第二帧起才上报读数。
- **单批缓存**：断网时只保留一个待发批次，缓存满后暂停采样，不保证断网期间连续采集。
- **重试预算**：每个批次最多重试 3 次，用完后暂停并保留数据；存储出错时停止采样，不自动格式化。
- **测试标记**：实测阶段保持 `XJ_IS_TEST_DATA=1`，所有上报会被平台标为测试数据。

## 与后端对接：设备协议 V1

设备只用一个接口上报所有数据：POST `/api/v1/device/ingest`，请求头带 `X-Device-ID`、`X-Device-Token` 和 `X-Experiment-Session-ID`。

一次请求是一个“批次”，里面可以包含多条记录：

| 记录类型                   | 内容                                             |
|------------------------|------------------------------------------------|
| <code>heartbeat</code> | 设备在线、运行时长等状态                                   |
| <code>reading</code>   | 传感器读数：指标名、数值、单位                                |
| <code>log</code>       | 事件日志，如 <code>DHT11_READ_FAILED</code> ，带级别和事件码 |

批次本身携带 `requestId`（全局唯一）、`bootId` + `sequenceNo`（本次启动内递增）、`uptimeMs`、固件版本和 `isTestData`。

服务端的处理规则：

- **幂等**：同一个 `requestId` 重发只返回原回执，不重复入库；固件断网重连后就是靠这一点安全重发。

- **时间质量**：设备没做 NTP 校时时不发 `occurredAt`，服务端按接收时间记录并标为 `server_fallback`。
- **限流与容量**：单批最多 100 条记录、请求体不超过 256KB、每分钟最多 120 次请求（都可配置）。
- **会话归属**：记录必须落在该会话的使用时间内才会挂到学生名下；归属不清的记录仍入库，但不向学生展示。

回执里每条记录都是 `accepted` 才算成功。`accepted` 只说明已保存，不代表学生一定看到。完整字段定义见 `docs/device-protocol.md`。

## 联调步骤

先用模拟器确认平台能收数，再接真机；每一步只改一个变量，出问题才好定位。

1. **启动平台**：在隔离环境（不要用正式业务库）`docker compose up -d --build`，用 `seed_demo` 创建测试学生、教师、设备和实验会话。
2. **模拟器试跑**：在 `simulator/` 安装后设置 `XINJIAN_API_BASE_URL`、设备 ID 和令牌，运行 `xinjian-simulator run normal --iterations 1`；在学生端“数据记录”里确认看到心跳和读数。再跑 `read-failure` 场景，确认教师端能看到异常。
3. **真机本地采集**：不配 `secrets.h` 烧录，串口里应看到启动版本、GPIO、首帧预热，之后每 3 秒一次温湿度或读取失败日志。
4. **真机接入平台**：确认 ESP32 能访问后端地址，写好 `secrets.h` 重新烧录；对照串口里的 `requestId` 和平台入库记录，只有服务端有记录才算到达。
5. **制造故障再恢复**：断电后只拔掉 DATA 线上电，累计读取失败后平台应给出异常和排查步骤；恢复接线，取得新读数后在学生端点“检查当前数据”。
6. **断网与重连**：断开实验 Wi-Fi，记录设备的待发批次；恢复后核对同一个 `requestId` 只入库一次。

## 实物验收与常见问题

T1—T4 四道门槛都留下可追溯的原始证据，才能写“内部技术功能验收通过”。

| 门槛        | 要证明什么         | 关键证据               |
|-----------|---------------|--------------------|
| T0 软件就绪   | 版本锁定、能编译      | 构建日志、固件与镜像 SHA-256 |
| T1 板上本地采集 | 真机能读到温湿度或如实报错 | 烧录与串口日志、接线照片       |

| 门槛       | 要证明什么            | 关键证据                 |
|----------|------------------|----------------------|
| T2 平台接入  | 数据真实入库并归属到会话     | requestId、回执、入库记录 ID |
| T3 异常与恢复 | 拔线能触发异常，恢复后能重新判断 | 前后诊断 ID、证据 ID        |
| T4 缓存与重连 | 断网不丢批次、重连不重复入库   | 本地批次状态、幂等记录          |

遇到问题先按现象对号：

| 现象                                 | 先查什么                                                    |
|------------------------------------|---------------------------------------------------------|
| 编译或镜像构建失败                          | 依赖是否取到、是否在仓库目录下；不要随意升级依赖                                |
| 无法烧录或串口没输出                         | USB 线是否支持数据、端口是否被占用、波特率 115200                          |
| 有心跳但没有温湿度                          | 是否还在首帧预热、是否因缓存满而暂停采样                                    |
| 串口报 <code>DHT11_READ_FAILED</code> | 报错阶段（超时/校验/量程），断电核对供电、DATA 和上拉电阻                        |
| 串口有记录，平台没有                         | 是否配了 <code>secrets.h</code> 、网络与地址可达、设备与会话是否对应、重试预算是否用完 |
| 数据已到，诊断仍是“未知”                      | 证据是否够触发规则（如窗口内累计 5 次失败）、时间质量是否可信                        |

读取失败不能单独证明是断线或器件损坏；也不要反复重启、格式化或换会话来“修复”上传问题。详细排查卡见内部实验指南第 7 节。

## 后续扩展：新传感器与新实验

新增一个实验不需要改平台代码，而是做两件事：写一个新的**实验资料包**，再让固件按协议 V1 上报对应的指标。GPIO LED 实验（`gpio_led_output 2.0.4`）就是第二个范例。

资料包放在 `backend/experiment_packages/<实验代号>/`，由 10 个数据文件组成：

| 文件                         | 写什么          |
|----------------------------|--------------|
| <code>metadata.yaml</code> | 实验名称、版本、来源登记 |

| 文件                                        | 写什么                     |
|-------------------------------------------|-------------------------|
| hardware.yaml                             | 器件、引脚、采样间隔、指标名与单位       |
| diagnosis/rules.yaml 、<br>fault_tree.yaml | 什么证据算异常，异常对应哪些有限候选和排查动作 |
| knowledge/                                | 概念解释与审核过的案例             |
| teaching/                                 | 分级排查步骤与提示               |
| tests/                                    | 正常与故障样例，发布前自动校验         |

约束：

- 资料包只允许数据，不能写脚本或任意表达式；规则只能用系统已注册的事实和比较运算。
- 服务端为每个文件和整个包计算 SHA-256，发布后是不可变快照；修改就升版本，历史诊断不会被改写。
- 阈值、引脚和案例先标测试草稿，经实测和教师确认后才能转为正式。
- 固件侧保持协议 V1 不变，只新增 metric\_key 和事件码；实测方法沿用 docs/hardware-validation-plan.md 的单因素试验。